



Binary Tree Path Sum (easy)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time complexity
 - Space complexity

Problem Statement

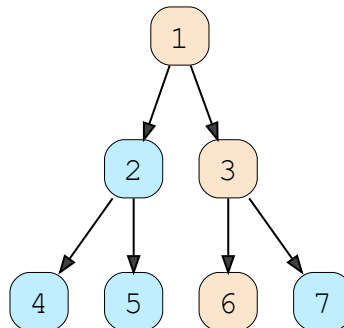
Given a binary tree and a number 'S', find if the tree has a path from root-to-leaf such that the sum of all the node values of that path equals 'S'.

Example 1:

S: 10

Output: true

Explanation: The path with sum '10' is highlighted



Example 2:

12



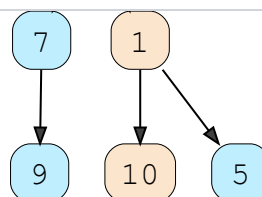
Output: true

Explanation: The path with sum '23' is highlighted

S: 16

Output: false

Explanation: There is no root-to-leaf path with sum '16'.



Tr



Try it yourself

Try solving this question here:

>

Java

Python3

JS

C++

```

1
2 class TreeNode {
3     int val;
4     TreeNode left;
5     TreeNode right;
6
7     TreeNode(int x) {
8         val = x;
9     }
10 };
11
12 class TreePathSum {
13     public static boolean hasPath(TreeNode root, int sum) {
14         // TODO: Write your code here
15         return false;
16     }
17
18     public static void main(String[] args) {
19         TreeNode root = new TreeNode(12);
20         root.left = new TreeNode(7);
21         root.right = new TreeNode(1);
22         root.left.left = new TreeNode(9);
23         root.right.left = new TreeNode(10);
24         root.right.right = new TreeNode(5);
25         System.out.println("Tree has path: " + TreePathSum.hasPath(root, 23));
26         System.out.println("Tree has path: " + TreePathSum.hasPath(root, 16));
27     }
28 }
29

```

Run

Save

Reset

Solution

As we are trying to search for a root-to-leaf path, we can use the **Depth First Search (DFS)** technique to solve this problem.

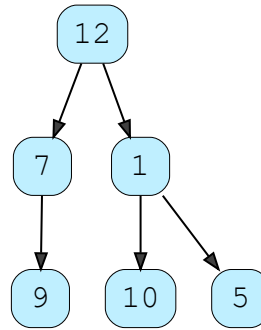
To recursively traverse a binary tree in a DFS fashion, we can start from the root and at every step, make two recursive calls one for the left and one for the right child.

Here are the steps for our Binary Tree Path Sum problem:

1. Start DFS with the root of the tree.
2. If the current node is not a leaf node, do two things:
 - Subtract the value of the current node from the given number to get a new sum => $S = S - \text{node.value}$
 - Make two recursive calls for both the children of the current node with the new number calculated in the previous step.
3. At every step, see if the current node being visited is a leaf node and if its value is equal to the given number. If both these conditions are true, we have found the required root-to-leaf path, therefore return **true**.
4. If the current node is a leaf but its value is not equal to the given number 'S', return false.

Let's take the example-2 mentioned above to visually see our algorithm:

S: 23



Let's start with the root node

1 of 10



Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```

1  class TreeNode {
2      int val;
3      TreeNode left;
4      TreeNode right;
5
6      TreeNode(int x) {
7          val = x;
8      }
9  };
10
11 class TreePathSum {
12     public static boolean hasPath(TreeNode root, int sum) {
13         if (root == null)
14             return false;
15
16         // if the current node is a leaf and its value is equal to the sum, we've found a path
17         if (root.val == sum && root.left == null && root.right == null)
18             return true;
19
20         // recursively call to traverse the left and right sub-tree
21         // return true if any of the two recursive call return true
22         return hasPath(root.left, sum - root.val) || hasPath(root.right, sum - root.val);
23     }
24
25     public static void main(String[] args) {
26         TreeNode root = new TreeNode(12);
27         root.left = new TreeNode(7);
28         root.right = new TreeNode(1);
29         root.left.left = new TreeNode(9);
30         root.right.left = new TreeNode(10);
31         root.right.right = new TreeNode(5);
32         System.out.println("Tree has path: " + TreePathSum.hasPath(root, 23));
33         System.out.println("Tree has path: " + TreePathSum.hasPath(root, 16));
34     }
  
```

```
35 }  
36
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ in the worst case. This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child).

[← Back](#)☒ Mark As Completed[Next →](#)[Introduction](#)[All Paths for a Sum \(medium\)](#)